VELLORE INSTITUTE OF TECHNOLOGY
School of Computer Science and Engineering
Computer Networks

PROJECT REPORT

AI-Adaptive Reliable File Transfer Protocol over UDP Using Machine Learning-Based Retransmission Timeout Prediction

A reliability layer built on plain UDP sockets, comparing TCP's RFC 6298 timeout, two machine-learning timeout estimators and RACK loss detection under identical, reproducible network conditions

Slot / Team	F1 / Team 10
Team members	Madhavarapu Venkata Lakshmi Jyothiradithya (24BCI0184) R. Sanjay (24BCI0181)
Programme	B.Tech Computer Science and Engineering
Course	Computer Networks
Submission	VTOP Assignment 8: Project Report

Abstract

UDP delivers datagrams without any guarantee: they can be lost, delayed, duplicated, reordered or corrupted. This project builds a complete reliable file-transfer protocol on top of plain UDP sockets in Python, using Selective Repeat with a 64-packet window, per-packet timers, selective and cumulative acknowledgements, CRC-32 on every packet, a SHA-256 check of the whole file and an atomic commit at the receiver. Its central question is how long a sender should wait before resending a packet. TCP answers with the RFC 6298 formula; we compare that baseline with two learned estimators (a random forest, v1, and a quantile gradient-boosting model, v2) and with RACK loss detection (RFC 8985). A custom impairment proxy decides the fate of every packet copy by hashing, so each strategy faces exactly the same network. Across 420 paired transfers of a 1 MiB file over seven network profiles, every file arrived intact and SHA-256-verified. A learned timeout on its own only trades speed against wasted retransmissions: v1 was 4.6% faster but caused 2.47× the unnecessary resends, and v2 was leaner (0.71×) but 2.2% slower. Combined with RACK, ML v2 was the only strategy that was both faster (+4.3% goodput) and less wasteful (0.88×) than TCP's formula. The work also shows that better offline prediction accuracy does not guarantee better behaviour on the wire.

Keywords: UDP, reliable transport, Selective Repeat ARQ, retransmission timeout, RFC 6298, RACK, machine learning, quantile regression, network emulation.

Contents

1. Introduction **4**
1.1 Background and motivation . . . 4
1.2 Problem statement . . . 4
1.3 Objectives . . . 4
1.4 Contributions . . . 4
2. Literature Review and Background **5**
2.1 UDP versus TCP . . . 5
2.2 Automatic repeat request (ARQ) protocols . . . 5
2.3 RTT estimation and RFC 6298 . . . 5
2.4 RACK loss detection (RFC 8985) . . . 5
2.5 Machine learning for transport protocols . . . 5
3. System Design **6**
3.1 Architecture . . . 6
3.2 Protocol workflow . . . 6
3.3 Packet format . . . 7
3.4 Integrity at three levels . . . 7
3.5 Failure handling . . . 7
4. Retransmission Timeout Strategies **8**
4.1 Conventional estimator (RFC 6298) . . . 8
4.2 Backoff with per-packet timers, and a bug found in testing . . . 8
4.3 Machine-learning estimator v1 . . . 8
4.4 Machine-learning estimator v2 . . . 9
4.5 RACK . . . 9
5. Network Simulation and Experimental Method **9**
5.1 Impairment proxy . . . 9
5.2 Experimental design . . . 10
5.3 Environment . . . 10
6. Results **10**
6.1 Reliability . . . 10
6.2 Overall comparison . . . 10
6.3 Per-network results . . . 11
6.4 Where the ML estimator helped . . . 12
6.5 Where it did not help . . . 12
7. Discussion **13**
7.1 Offline accuracy did not predict online behaviour . . . 13
7.2 Why the combination works . . . 13
7.3 What the model learned . . . 14
7.4 Generalisation . . . 14
7.5 Cost . . . 14
8. Testing **14**
9. Security Considerations **14**
10. Installation and Execution **15**
11. Limitations and Future Work **15**
11.1 Limitations . . . 15
11.2 Future work . . . 15
12. Conclusion **16**
References **16**

1. Introduction

1.1 Background and motivation

TCP gives applications a reliable byte stream, but it is built into the operating system and its behaviour is hard to change or observe. UDP is the opposite: a thin service that sends independent datagrams with no acknowledgements, ordering, retransmission or flow control. Modern transports such as QUIC are built on UDP precisely so that reliability can be implemented, tuned and studied in user space.

Any reliable protocol must decide when a missing acknowledgement means a lost packet. This retransmission timeout (RTO) is a balance. If it is too short, packets that were only delayed are resent, wasting bandwidth on a network that is already struggling (spurious retransmissions). If it is too long, a genuinely lost packet leaves the sender idle and, with a sliding window, stalls new data. TCP sets the RTO with the Jacobson/Karels smoothing formula standardised in RFC 6298. Machine learning offers an alternative: a model that looks at recent round-trip-time (RTT) history and predicts how late the next packets are likely to be.

1.2 Problem statement

Design and implement a client-server file-transfer application over UDP that guarantees complete, ordered, duplicate-free and bit-identical delivery, and evaluate whether a machine-learning RTO estimator performs better than the conventional RFC 6298 estimator under varied, reproducible network conditions.

1.3 Objectives

1. Build a reliability layer over UDP that detects and recovers from loss, corruption, duplication and reordering, and verifies the received file with SHA-256.
2. Implement the RFC 6298 RTO estimator with Karn's algorithm and exponential backoff as the baseline.
3. Train machine-learning models that predict a suitable RTO from features the sender can observe, and run them safely inside the protocol.
4. Build a portable network-impairment simulator that gives every strategy exactly the same network.
5. Compare the strategies statistically on goodput, retransmissions, wasted retransmissions and recovery delay, and explain the results.
6. Provide a command-line dashboard and a browser-based live lab to demonstrate the system.

1.4 Contributions

- A working, tested reliable transport over UDP (Selective Repeat, 24-byte header, per-packet and per-file integrity checks, optional HMAC authentication), runnable with the Python standard library alone.
- Six timeout strategies evaluated head to head: RFC 6298 (K = 4), an aggressive RFC variant (K = 2), ML v1, ML v2, RFC + RACK, and ML v2 + RACK.
- A hash-keyed impairment proxy that makes paired, reproducible experiments possible, with ground-truth counts of unnecessary retransmissions.
- 420 paired transfers with exact Wilcoxon tests, plus offline evaluation of 25,721 timeout decisions on held-out runs.
- 136 automated tests covering packet fuzzing, every single-bit flip, RTO mathematics, RACK timing, crashes and path-traversal attempts.

2. Literature Review and Background

2.1 UDP versus TCP

Property	UDP (RFC 768)	TCP
Connection	None	Three-way handshake
Delivery and ordering	Not guaranteed	Guaranteed
Retransmission	None	Timeout and fast retransmit
Flow / congestion control	None	Receive window / congestion window
Header	8 bytes	20–60 bytes
Where reliability lives	Application (this project)	Operating system kernel

Table 1: UDP and TCP compared

2.2 Automatic repeat request (ARQ) protocols

Stop-and-wait keeps one packet in flight, so throughput is limited to one payload per RTT (about 4 KB/s for 1,200 bytes on a 300 ms path). **Go-Back-N** keeps N packets in flight with one timer, and on a loss resends the whole window. **Selective Repeat** keeps N packets in flight, buffers out-of-order packets at the receiver and resends only the missing ones, each with its own timer. Selective Repeat was chosen because it keeps a high-latency pipe full, tolerates reordering, and makes every retransmission correspond to a single timeout decision, which is exactly what this project needs to measure.

2.3 RTT estimation and RFC 6298

Jacobson (1988) proposed tracking both a smoothed RTT (SRTT) and its mean deviation (RTTVAR), which RFC 6298 standardises:

```
first sample R:   SRTT = R,  RTTVAR = R/2
later samples R:  RTTVAR = (1 - 1/4) * RTTVAR + 1/4 * |SRTT - R|
                  SRTT   = (1 - 1/8) * SRTT   + 1/8 * R
RTO = clamp(SRTT + max(G, K * RTTVAR), RTO_min, RTO_max)        K = 4
on timeout: RTO = RTO * 2  (exponential backoff, until a new valid sample)
```

Karn's algorithm (Karn and Partridge, 1987) adds that no RTT sample may be taken from a retransmitted packet, because its acknowledgement cannot be attributed to a particular copy.

2.4 RACK loss detection (RFC 8985)

Waiting for a timer is the slow way to find a loss. RACK (Recent ACKnowledgment) declares packet P lost if a packet sent after P has already been delivered and more than one RTT plus a reordering window has passed since P was sent. Because it reasons about time rather than counting duplicate acknowledgements, it tolerates moderate reordering.

2.5 Machine learning for transport protocols

Learned congestion control (for example Remy, Winstein and Balakrishnan, 2013) showed that machine-generated rules can outperform hand-designed ones within the conditions they were trained for. Tree ensembles such as random forests (Breiman, 2001) and gradient boosting (Friedman, 2001) suit this setting: they are accurate on tabular features, cheap to evaluate, and can be exported as plain data. Their known weakness, that they do not extrapolate beyond the training range, turns out to matter in this project (Section 7.4).

3. System Design

3.1 Architecture

The system runs as three independent processes. The sender and receiver contain no simulation logic, so the same programs work over a real link; all network impairment is done by a separate proxy placed between them.

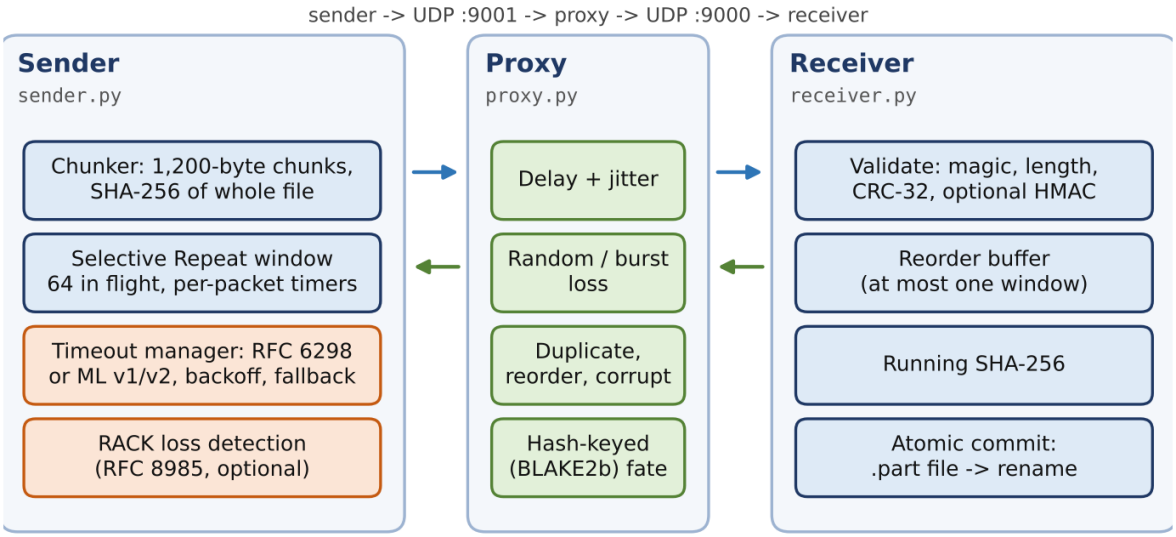


Figure 1: System architecture. Data flows sender → proxy → receiver; acknowledgements flow back.

Module	Responsibility
protocol/packet.py	Header layout, encode/decode, CRC-32, optional HMAC, validation
protocol/window.py	Send window (timers, retransmit counts, RACK) and receive buffer
protocol/reliability.py	Sender and receiver state machines: handshake, data, FIN
timeout/conventional.py	RFC 6298 SRTT / RTTVAR / RTO
timeout/features.py	ML feature vector, shared by training and runtime
timeout/ml_estimator.py	JSON model loading, pure-Python prediction, safety policy
timeout/manager.py	Chooses the RTO; Karn-aware backoff identical for every mode
simulation/	Network profiles, deterministic impairment, the UDP proxy
training/	Dataset generation, training, calibration, offline evaluation
experiments/	Paired trials, statistics, plots, results tables
sender.py, receiver.py, proxy.py, demo.py, lab_server.py	Command-line programs and the live lab

Table 2: Module overview

3.2 Protocol workflow

1. **Handshake.** The sender picks a random 32-bit transfer ID, computes the file's SHA-256 and sends SYN with the size, chunk count, chunk size, window, hash and filename. The receiver validates it, creates a temporary *.part* file exclusively and replies SYN_ACK (or ERROR with a reason).
2. **Data.** Up to 64 unacknowledged chunks are in flight. Each records its send time, retransmission count, timer deadline and backoff epoch.
3. **Acknowledgement.** One ACK per DATA packet carries the chunk being acknowledged (selective) and the lowest chunk not yet received (cumulative), so a lost ACK is harmless.
4. **Timeout.** The sender blocks on the socket only until the earliest timer deadline (no busy-waiting). An expired packet is resent and the timeout manager may back off. More than 15 retries aborts the transfer.

5. **Finish.** After every chunk is acknowledged the sender sends FIN. The receiver checks completeness and the SHA-256, calls fsync, renames the *.part* file atomically and replies FIN_ACK(OK); otherwise it deletes the partial file and reports HASH_MISMATCH or INCOMPLETE. Only FIN_ACK(OK) counts as success.

3.3 Packet format

Every packet starts with a fixed 24-byte big-endian header (struct format `!HBBBBHIIII`).

Offset	Size	Field	Meaning
0	2	magic	0x5255 ("RU")
2	1	version	1
3	1	msg_type	1 SYN, 2 SYN_ACK, 3 DATA, 4 ACK, 5 FIN, 6 FIN_ACK, 7 ERROR
4	1	flags	bit 0 = HMAC tag present
5	1	reserved	must be 0
6	2	payload_len	payload bytes (≤ 1,200)
8	4	transfer_id	random per transfer
12	4	seq	DATA: chunk index; ACK: chunk acknowledged
16	4	ack	cumulative: lowest chunk not yet received
20	4	checksum	CRC-32 over header, payload and tag
24	≤ 1,200	payload	message-specific
24+len	16	tag	optional HMAC-SHA256 (truncated)

Table 3: Packet header layout

The 1,200-byte payload keeps the largest datagram at 1,268 bytes including UDP and IPv4 headers, below the 1,280-byte minimum MTU of IPv6, so packets are never fragmented. Header overhead is 2%.

3.4 Integrity at three levels

Check	Where	Covers	Purpose
UDP checksum	Kernel	UDP header and data	Drops datagrams corrupted in transit
CRC-32	Application, per packet	Header, payload, tag	Detects accidental corruption, including corruption injected by the proxy
SHA-256	Application, per file	Whole file	Proves the received file is bit-identical
HMAC-SHA256 (optional)	Application, per packet	Header and payload	Authenticates the sender with a pre-shared key

Table 4: Integrity mechanisms

3.5 Failure handling

Situation	Behaviour
ACK lost	A later cumulative ACK covers it, or the packet is resent and re-acknowledged
Duplicate DATA	Re-acknowledged, not written twice
Out-of-order DATA	Buffered (at most one window) until the gap is filled
Corrupted packet	Fails CRC-32; dropped and counted; the sender's timer recovers it
Lost FIN_ACK	Finished transfers linger 5 s and answer a repeated FIN
Sender crashes	Receiver idle timeout (30 s) deletes the .part file
Receiver crashes	Sender exceeds the retry limit and exits with an error code

Table 5: Failure handling

4. Retransmission Timeout Strategies

4.1 Conventional estimator (RFC 6298)

The baseline follows RFC 6298 exactly, with α = 1/8, β = 1/4, K = 4 and 1 ms clock granularity. The one deliberate deviation is RTO_min = 30 ms instead of the RFC's 1 s, because on our 20–300 ms simulated paths a 1 s floor would make every estimator output the same value and hide every difference. The implementation is short and explicit:

```
def update(self, rtt: float) -> float:
    """Feed one valid (Karn-filtered) RTT sample in seconds; returns new RTO."""
    if self.srtt is None:
        self.srtt = rtt
        self.rttvar = rtt / 2
    else:
        self.rttvar = (1 - self.beta) * self.rttvar + self.beta * abs(self.srtt - rtt)
        self.srtt = (1 - self.alpha) * self.srtt + self.alpha * rtt
    self.samples += 1
    self.rto = self._clamp(self.srtt + max(self.granularity, self.k * self.rttvar))
    return self.rto
```

4.2 Backoff with per-packet timers, and a bug found in testing

With 64 timers, one network outage makes many timers fire together. If each doubled the RTO, it would jump straight to the 10 s maximum. The timeout manager therefore keeps a **backoff epoch** that changes whenever the backoff level changes; a timeout raises the level only if that packet was sent in the current epoch, so the RTO doubles once per round of timeouts, as with TCP's single timer.

```
def on_timeout(self, now: float, epoch_at_send: int) -> None:
    self.timeouts += 1
    self.history.add_timeout(now)
    if epoch_at_send == self.epoch and self.backoff_level < MAX_BACKOFF_LEVEL:
        self.backoff_level += 1
        self.epoch += 1
        self.backoff_events += 1
```

The first version compared backoff levels instead of epochs. A packet resent during an earlier backoff could push the level up again after it had been reset, and on the lossy profile the RTO climbed to 1.9 s on a 50 ms path. Logging the RTO trace exposed the problem; switching to epochs and re-arming all timers when a backoff ends cut a 3 MB transfer from 22 s to 4.3 s and loss recovery from about 460 ms to 60 ms.

4.3 Machine-learning estimator v1

Target. At each RTT sample at time t, the model predicts the 95th percentile of the RTTs of packets sent in the next 0.5 s. An RTO is a threshold, so a percentile gives an explicit budget: about 5% of delivered packets would outlive their timer. The future is used only to build labels offline; features use only the past.

Feature	Meaning	Why it helps
log_srtt	log of smoothed RTT	Absolute scale of the path
rttvar_rel	RTTVAR / SRTT	What the formula believes about variability
last, mean, std, p90, max, min (/ SRTT)	Shape of the last 16 RTTs	Heavy tails that mean + 4·RTTVAR misjudges
slope	Least-squares trend of RTT	Is the path getting slower right now?
retx_ratio	Resends among the last 64 sends	Signals lossy periods
since_timeout_rel	Time since last timeout / SRTT	Recent instability
n_samples	History available (≤ 16)	Confidence of the other features

Table 6: The 12 features, all observable by the sender

Data and leakage control. 224 real 1 MiB transfers (7 profiles × 32 seeds) were logged under the conventional estimator. Data were split by run, never by row: seeds 0–19 for training, 100–105 for validation,

200–205 for testing, and 1000+ for end-to-end experiments. A random row split would have leaked nearly identical neighbouring rows into the test set. v1 is a 40-tree random forest; its safety margin was calibrated on validation runs so that it is never more spurious than the formula offline.

Safe deployment. The model is stored as JSON (tree arrays and metadata), never pickle, because unpickling can execute code. It is evaluated in pure Python, at most every 10 ms, so the sender needs no scikit-learn at runtime. A safety policy wraps every prediction:

```
mode ml:  no model, fewer than 8 samples, error or NaN  -> use rto_conv (fallback, counted)
          proposal = margin * exp(prediction)
          lo = max(RTO_min, 0.5 * rto_conv, largest of the last 16 RTTs)
          hi = min(RTO_max, 2.0 * rto_conv)
          rto = clamp(proposal, lo, hi)
both:     final = min(RTO_max, rto * 2 ** backoff_level)        # identical backoff
```

4.4 Machine-learning estimator v2

Problem measured in v1	Change in v2
Squared-error loss treats too-short and too-long timeouts alike	Quantile (pinball) loss: under-prediction costs q/(1−q) times more
Trained only on traces recorded under RFC 6298 (covariate shift)	Second collection round with v1 in control (182 transfers)
Margin chosen offline; online waste doubled	Quantile and margin chosen on 147 real validation transfers with a rule fixed in advance: waste ≤ 1.15 × RFC, then the best goodput

Table 7: From v1 to v2

v2 is a 300-tree gradient-boosting model of depth 4 with the same 12 features, label and safety policy, so the comparison isolates the training changes.

4.5 RACK

A first attempt at fast loss detection, the classic "three later ACKs" rule, fired about 3,400 false resends per transfer under per-packet jitter, so it was replaced with RACK. Its reordering window starts at a quarter of the minimum RTT, grows each time a RACK resend proves unnecessary, and resets after 16 clean repairs. A RACK resend is not a timeout: it causes no backoff, and Karn's rule still applies.

5. Network Simulation and Experimental Method

5.1 Impairment proxy

macOS has no *tc netem*, so impairment is done by a portable user-space UDP proxy. It adds delay, jitter, random and burst loss, duplication, reordering, corruption and delayed ACKs. Crucially, the fate of every packet copy is derived from `BLAKE2b(seed, direction, type, seq, copy)`. With a shared random stream, one extra retransmission would shift every later random draw and two strategies would face different networks; with hashing, "the second copy of chunk 117 is lost" is true in every run.

Profile	One-way delay	Jitter	Loss (data / ACK)	Other
stable	10 ms	±1 ms	0 / 0	–
high_latency	150 ms	±5 ms	0.5% / 0.5%	–
lossy	25 ms	±2.5 ms	5% / 2%	–
bursty_loss	25 ms	±2.5 ms	0.5% + bursts of 50%	bad periods ≈ 50 ms every 0.8 s
jitter_heavy	30 ms	Pareto, up to 160 ms	1% / 1%	reordering emerges
changing	15 → 120 → 30 ms	2–10 ms	1–3%	5 s cycle of ramps and steps
dup_reorder	20 ms	±5 ms	1% / 1%	2% duplicated, 5% reordered, 1% corrupted

Table 8: Network profiles

5.2 Experimental design

- **Trials:** 7 profiles × 10 seeds (1000–1009) × 6 strategies = 420 transfers of a 1 MiB file, disjoint from all training seeds.
- **Pairing:** for each profile and seed, every strategy meets the same network; run order alternates between seeds.
- **Statistics:** mean, standard deviation, paired difference with a 95% bootstrap confidence interval, and an exact two-sided Wilcoxon signed-rank test. With 10 pairs the smallest attainable p-value is about 0.002.
- **Metrics:** goodput (file bytes / duration), retransmissions, timeouts, **unnecessary retransmissions** and **loss-recovery delay** from the proxy's ground truth, SHA-256 (verified by the receiver and re-hashed independently), sender CPU and memory.

5.3 Environment

Item	Value
Platform	macOS, single machine, loopback interface
Language	Python 3.9+; runtime uses only the standard library
Libraries (training, experiments)	NumPy, pandas, scikit-learn, Matplotlib; pytest optional
Tools	Visual Studio Code, Git
Network simulation	Custom user-space UDP impairment proxy (proxy.py)

Table 9: Software and environment

6. Results

6.1 Reliability

All 420 transfers completed successfully, and all 420 received files matched the original SHA-256, both as reported by the receiver and when re-hashed independently by the test harness. Measured loss rates per profile were identical across strategies (for example 5.6% on *lossy* for every mode), confirming that the hash-keyed proxy gave each strategy the same network.

6.2 Overall comparison

Strategy	Goodput vs RFC 6298	Unnecessary resends vs RFC 6298	Character
RFC 6298 (K = 4)	baseline	1.00×	Reference
ML v1 (random forest)	+4.6%	2.47×	Faster, wasteful
RFC with K = 2	+6.6%	3.30×	Faster, most wasteful
ML v2 (quantile boosting)	−2.2%	0.71×	Lean, slower
RFC + RACK	+4.9%	1.30×	Faster, a little more waste
ML v2 + RACK	**+4.3%**	**0.88×**	**Faster and leaner**

Table 10: Averages over the six lossy profiles, paired by seed

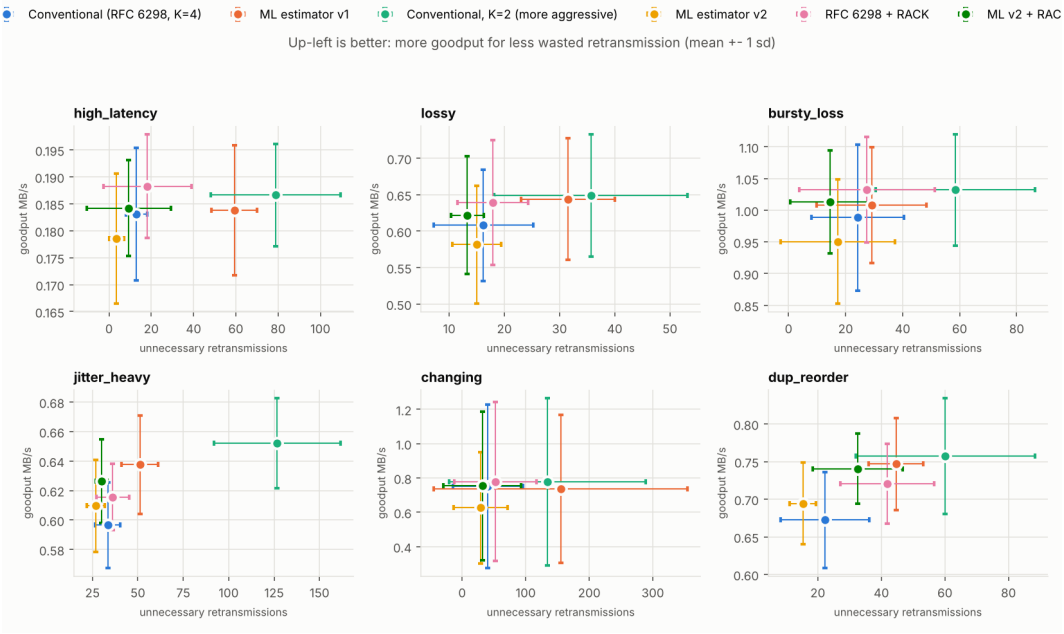


Figure 2: Goodput change against unnecessary retransmissions, both relative to RFC 6298. Top-left is better.

A learned timeout on its own slides along a single trade-off line: v1 sits next to the simply shorter K = 2 formula (fast and wasteful), and v2 sits at the lean, slower end. RACK moves the frontier, and ML v2 + RACK is the only point that is better than the baseline on both axes.

6.3 Per-network results

Profile	RFC 6298	ML v1	K = 2	ML v2	RFC + RACK	ML v2 + RACK
stable	3.079 / 0	3.081 / 0	3.078 / 0	3.027 / 0	3.022 / 9	2.991 / 8
high_latency	0.183 / 13	0.184 / 59	0.187 / 79	0.179 / 4	0.188 / 18	0.184 / 9
lossy	0.608 / 16	0.644 / 32	0.649 / 36	0.581 / 15	0.639 / 18	0.622 / 13
bursty_loss	0.989 / 24	1.008 / 29	1.032 / 58	0.951 / 17	1.032 / 28	1.013 / 15
jitter_heavy	0.597 / 34	0.638 / 51	0.652 / 127	0.610 / 27	0.616 / 36	0.627 / 30
changing	0.753 / 41	0.737 / 155	0.777 / 134	0.626 / 29	0.778 / 52	0.754 / 32
dup_reorder	0.672 / 22	0.747 / 45	0.757 / 60	0.694 / 15	0.721 / 42	0.741 / 33

Table 11: Mean goodput (MB/s) / unnecessary retransmissions per 1 MiB transfer, 10 seeds each

Profile	ML v1 vs RFC	ML v2 + RACK vs RFC	RFC + RACK vs RFC
high_latency	+0.4%* / +356%	+0.6% / −28%	+2.9% / +40%
lossy	+5.9%* / +94%	+2.3%* / −18%	+5.1%* / +10%
bursty_loss	+2.0% / +20%	+2.4% / −40%	+4.4%* / +14%
jitter_heavy	+6.9%* / +53%	+5.0%* / −10%	+3.2%* / +8%
changing	−2.1% / +282%	+0.2% / −22%	+3.4%* / +29%
dup_reorder	+11.1%* / +100%	+10.2%* / +46%	+7.2%* / +87%

*Table 12: Paired change in goodput / unnecessary retransmissions. * = significant goodput difference (Wilcoxon $p < 0.05$)*

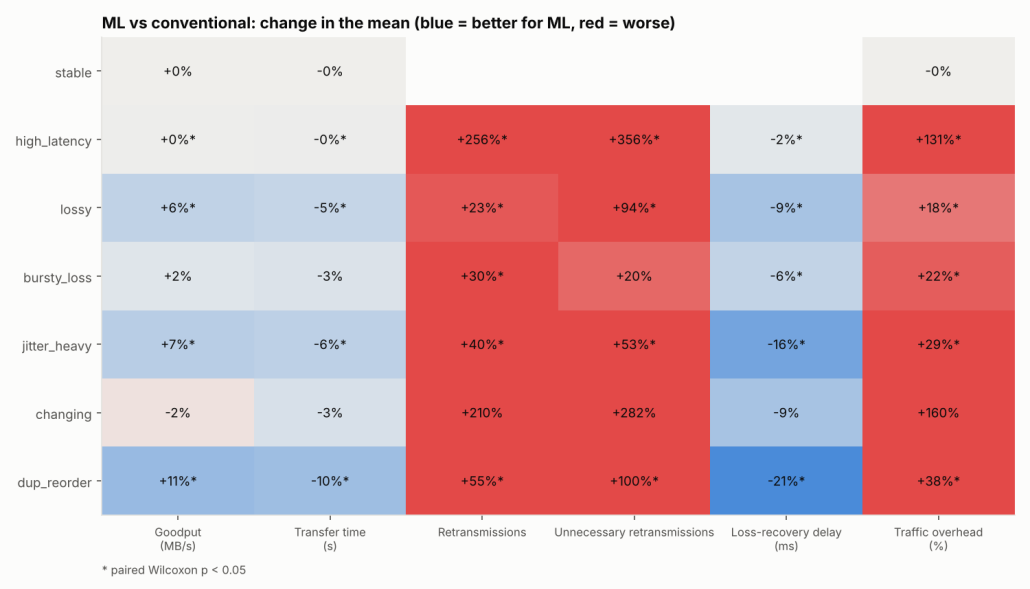


Figure 3: Goodput change of each strategy against RFC 6298 on each network.

6.4 Where the ML estimator helped

Under heavy-tailed jitter and reordering the formula's 4 × RTTVAR term is inflated by each delay spike for many samples afterwards, so after a real loss the sender waits much longer than necessary. The model sees the shape of recent RTTs and proposes a timeout close to the real upper tail. Offline, its error on *jitter_heavy* was 6.4 ms against the formula's 12.2 ms, and end-to-end, loss recovery was 16% faster and goodput 6.9% higher (p = 0.002).

The ablation against K = 2 shows that most of v1's speed-up is simply a shorter timeout, but not all of it. At similar goodput, v1 caused 60% fewer unnecessary retransmissions than K = 2 under heavy jitter (p = 0.002) and 50% fewer under burst loss (p = 0.045). There, the model shortens the timeout selectively rather than everywhere, which a fixed K cannot express.

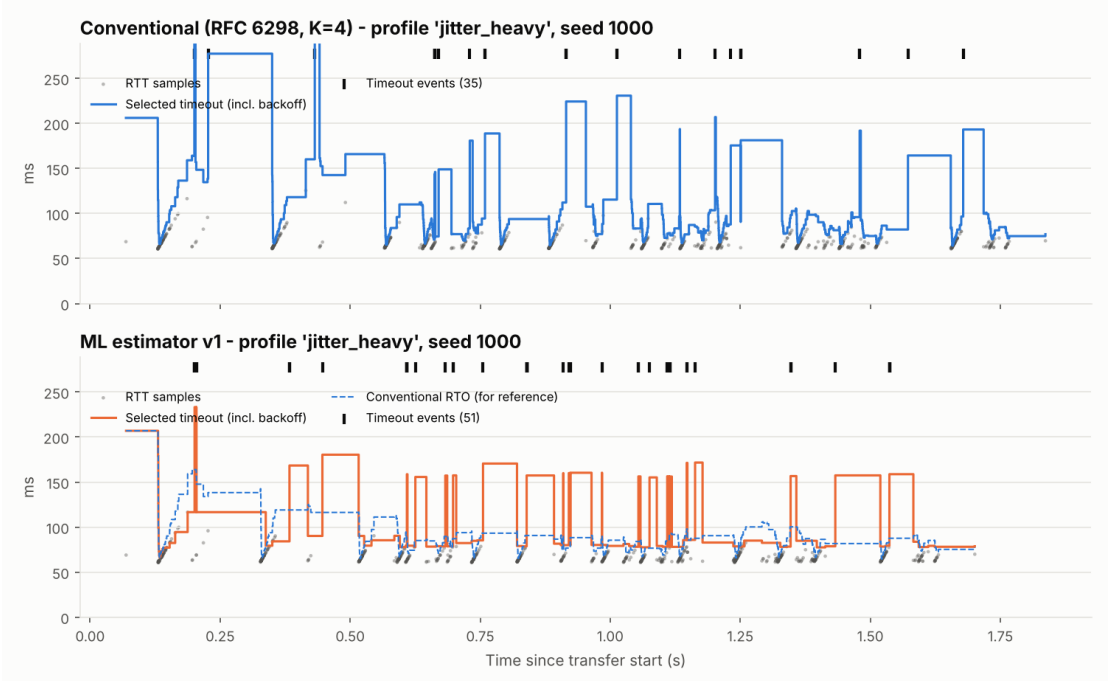


Figure 4: RTT samples and the RTO chosen by each estimator on the jitter_heavy network.

6.5 Where it did not help

- **stable:** both estimators sit at the 30 ms floor and no timeouts occur, so the estimator is irrelevant.

- **high_latency:** throughput is limited by the 64-packet window over a 300 ms RTT, so a shorter RTO barely changes goodput (+0.4%) while v1 produced 4.6× the unnecessary retransmissions.
- **changing:** v1 gave −2.1% goodput and 3.8× the waste. While delay ramps up, newly sent packets take longer than anything in the recent history, and their timers fire just before their ACKs arrive (Figure 5).

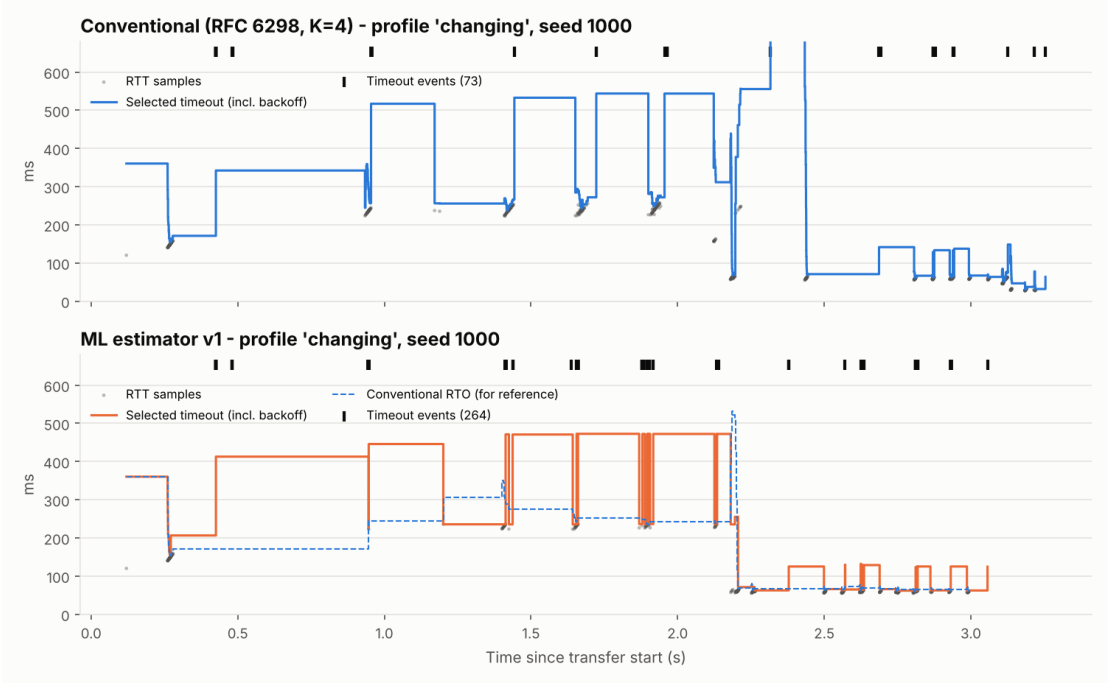


Figure 5: RTT and RTO on the changing network. Clusters of early timeouts appear as delay ramps up.

7. Discussion

7.1 Offline accuracy did not predict online behaviour

Measure	RFC 6298	ML v1
Offline mean absolute error (held-out runs)	10.0 ms	5.7 ms
Offline share of packets outliving the timer	7.2%	5.1%
Online unnecessary retransmissions (relative)	1.00×	2.47×

Table 13: v1 was better offline on both measures, then worse on the wire

This is the most important lesson of the project. Offline, each decision is scored independently. Online, delays are correlated: when a slow period starts, every in-flight packet's timer is about to fire, so one slightly short timeout becomes dozens of retransmissions at once. Each spurious timeout also removes an RTT sample (Karn), thinning the history exactly when conditions change. And because v1 was trained on traces recorded under the formula, it met conditions under its own control that it had never seen (covariate shift). v2 addressed each of these causes.

7.2 Why the combination works

RACK repairs most losses from the acknowledgement pattern without waiting for any timer. The RTO then only has to cover the cases RACK cannot see, such as the last packets of a window. There, v2's deliberate caution costs little time and avoids spurious resends. The protocol fix and the learned timeout are therefore complementary rather than competing.

7.3 What the model learned

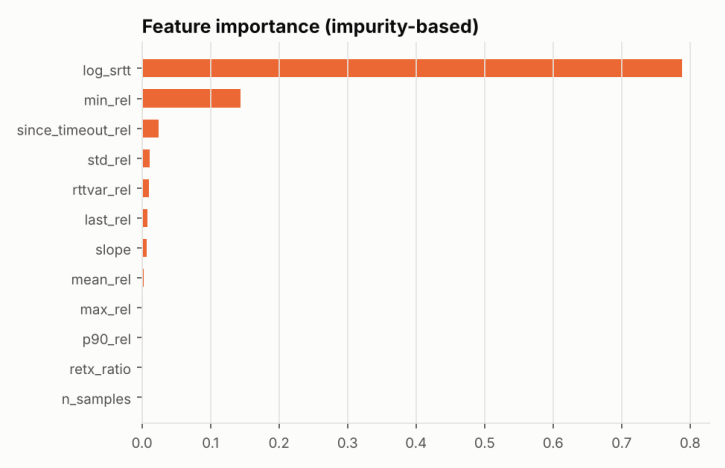


Figure 6: Feature importances of the v1 random forest.

log_srtt dominates (about 0.79) and *slope* is almost unused: the model mostly learned "the typical good timeout for this RTT level" rather than "the RTT is rising, so wait longer".

7.4 Generalisation

A model trained without the *changing* profile had a higher error on it than the plain formula (25.3 ms against 18.9 ms). Tree ensembles do not extrapolate to dynamics outside their training data, which is why the safety band [0.5×, 2×] around the formula and the automatic fallback are essential for deployment.

7.5 Cost

Sender CPU per second of transfer was 0.039 s for RFC 6298, 0.053 s for ML v2, 0.064 s for RACK and 0.076 s for both together, under 8% of one core. On the loss-free *stable* network, where the Python sender is CPU-bound, this costs about 2% goodput; a compiled implementation would remove most of it.

8. Testing

Test file	Tests	What is covered
test_packet.py	32	Encoding and validation, fuzzing, every single-bit flip detected
test_timeout.py	30	RFC 6298 equations, Karn, backoff epochs, ML fallback and safety bounds, model-file validation
test_window.py	25	Send window, timers, re-arming, RACK timing, receive buffer limits
test_reliability.py	23	End-to-end under loss, bursts, duplication, reordering, corruption, crashes, idle senders, hash mismatch, path traversal, HMAC
test_impairment.py	11	Determinism and statistics of the simulator
test_file_integrity.py	7	Empty, large and binary files
test_metrics.py	4	Metric definitions
test_lab_server.py	4	Live-lab request validation
Total	**136**	

Table 14: Automated test suite (python3 -m unittest discover -s tests -t .)

9. Security Considerations

- **Single validation boundary:** every datagram is parsed in one function with struct and checked for magic, version, flags, length consistency and CRC before anything else touches it. Malformed input raises an error and is dropped; it never crashes the receiver.

- **Safe filenames:** empty names, names over 255 bytes, '.', '..', slashes, backslashes and control characters are rejected; files are created exclusively inside the output directory, after checking the real path.
- **Bounded resources:** maximum file size 1 GiB, window 1,024, 8 concurrent transfers, reorder buffer ≤ window, 1,241-byte reads, idle timeouts and retry limits.
- **No unsafe deserialisation:** the ML model is JSON, checked node by node on load; pickle is never used.
- **Optional HMAC-SHA256** with a pre-shared key read from a file (never the command line, which other users can see). Finished transfer IDs are refused for 10 minutes as simple replay protection.
- **Scope:** there is no encryption and no congestion control. This is an academic protocol, not a replacement for TCP, QUIC or SFTP.

10. Installation and Execution

```
# setup (macOS / Linux, Python 3.9+)
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# one-command demo with live dashboard
python3 demo.py --profile lossy
python3 demo.py --profile changing --timeout-mode ml
python3 demo.py --profile jitter_heavy --compare

# manual run in three terminals
python3 receiver.py --host 0.0.0.0 --port 9000 --output-dir received_files
python3 proxy.py --listen-port 9001 --target-port 9000 --profile lossy --seed 1 --status
python3 sender.py --file sample.pdf --host 127.0.0.1 --port 9001 --timeout-mode ml --dashboard

# live lab in the browser, then open http://localhost:8000
python3 lab_server.py

# tests
python3 -m unittest discover -s tests -t . -v
```

11. Limitations and Future Work

11.1 Limitations

- All experiments ran on one machine over loopback through a user-space proxy: no real queueing, bandwidth limit or cross traffic, and timing accurate to about 1 ms.
- The models were trained and tested on the same simulator; performance on real networks is unproven.
- Ten seeds per profile detect large effects reliably but are weak for differences of 1–2%.
- The three version-2 strategies were run after the original three rather than interleaved with them, so slow drift in machine load could shift comparisons between the two groups by a percent or two.
- There is no congestion control, so the protocol is not fair to other traffic on a shared link.

11.2 Future work

- Run the same paired experiments over real links and Linux tc/netem.
- Add congestion control (AIMD) and measure its interaction with the learned timeout.
- Train on real-network traces and on more varied dynamics, or add features that emphasise trends, to address the *changing* profile.
- Port the sender to a compiled language to remove the CPU cost of model evaluation.

12. Conclusion

The project delivers a reliable file-transfer protocol over UDP that worked in every one of 420 trials across seven hostile network conditions, with every file verified bit for bit. On the research question, the answer is nuanced. A learned retransmission timeout on its own does not beat TCP's formula: it only moves along the trade-off between speed and wasted retransmissions, and a model that is more accurate offline can be more wasteful on the wire. Combined with RACK loss detection, however, a cautious quantile model was the only strategy that was both faster (+4.3% goodput) and less wasteful (0.88× unnecessary resends) than RFC 6298. Machine learning helps most when it refines a sound protocol mechanism rather than replacing it.

References

[1]	Postel, J. (1980). User Datagram Protocol. RFC 768, IETF.
[2]	Paxson, V., Allman, M., Chu, J. and Sargent, M. (2011). Computing TCP's Retransmission Timer. RFC 6298, IETF.
[3]	Cheng, Y., Cardwell, N., Dukkipati, N. and Jha, P. (2021). The RACK-TLP Loss Detection Algorithm for TCP. RFC 8985, IETF.
[4]	Jacobson, V. (1988). Congestion avoidance and control. *Proc. ACM SIGCOMM '88*, 314–329.
[5]	Karn, P. and Partridge, C. (1987). Improving round-trip time estimates in reliable transport protocols. *Proc. ACM SIGCOMM '87*, 2–7.
[6]	Winstein, K. and Balakrishnan, H. (2013). TCP ex Machina: Computer-generated congestion control. *Proc. ACM SIGCOMM 2013*.
[7]	Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
[8]	Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
[9]	Krawczyk, H., Bellare, M. and Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. RFC 2104, IETF.
[10]	Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.
[11]	Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
[12]	Kurose, J. F. and Ross, K. W. *Computer Networking: A Top-Down Approach*. Pearson.